



Documentación

de

Práctica MuleSoft Trainee

Elaborado por:
Enrique Gamboa Hernández

SPS Solutions

8 de mayo de 2026

Contenido

Introducción y problemática	4
Acerca de la Base de Datos.....	4
Procedimiento	5
Extras	24
APIManager.....	24
API Discovery.....	25
ClientID Police Enforcement.....	28

Índice de Imágenes

Imagen 1. Creación de un nuevo Mule Project
Imagen 2. New Mule Configuration File
Imagen 3. Menú de src/main/mule
Imagen 4. Archivos de propiedades creados
Imagen 5. Menú de creación de archivos de propiedades
Imagen 6. HTTP Listener configuración
Imagen 7. Menú para crear Configuration properties
Imagen 8. Configuración del Global Element Properties
Imagen 9. HTTP Listener y Configuration Properties de spscustomersapi a global
Imagen 10. Propiedades local y dev
Imagen 11. Configuración dinámica de HTTP Listener
Imagen 12. Configuración de Global Property
Imagen 13. Archivos dev/local.secure.properties
Imagen 14. Importación del módulo Mule Secure Configuration Properties
Imagen 15. Menú para añadir Secure Properties Config
Imagen 16. Configuración del Secure Properties Config
Imagen 17. Cifrado de contraseña de acceso a la base de datos
Imagen 18. Variables de acceso a la base de datos
Imagen 19. Dependencia Database Connector en Exchange
Imagen 20. Database Config en Global Elements de global.xml
Imagen 21. Database Config añadiendo controlador para la conexión
Imagen 22. Configuración de la conexión a base de datos
Imagen 23. Cambio provisional de secure.key
Imagen 24. Test de conexión a la base de datos
Imagen 25. Implementación de SELECT dentro del Flow principal
Imagen 26. Configuración de componente Select
Imagen 27. Componente Transform Message agregado al Flow
Imagen 28. Configuración reestablecida de Secure Properties
Imagen 29. Run Configurations para probar localmente
Imagen 30. Test de llamada a la API antes de despliegue a Cloudhub
Imagen 31. Despliegue a Cloudhub desde la web seleccionando archivo
Imagen 32. Despliegue exitoso a Cloudhub.
Imagen 33. Llamada GET al endpoint de customers exitosa
Imagen 34. Nombre de API Specification
Imagen 35. Menú de crear una especificación nueva.
Imagen 36. API Designer lleno
Imagen 37. Tipo de dato Customer lleno

Imagen 38. Creación del recurso /customers
Imagen 39. Recurso /customers con la configuración llena
Imagen 40. Test con Mocking Service
Imagen 41. Creación de nueva API dentro de API Manager
Imagen 42. Selección de API Specification realizada anteriormente
Imagen 43. Review de API nueva
Imagen 44. API Summary de SPS Customers API ya creada
Imagen 45. API ID agregado a nuestros archivos de propiedades
Imagen 46. Global Type en el menú
Imagen 47. Configuración del elemento global API Autodiscovery
Imagen 48. Test con cliente REST de aplicación con Autodiscovery
Imagen 49. Archivo mule-artifact.json
Imagen 50. Propiedades añadidas antes de despliegue a Cloudhub con API Autodiscovery
Imagen 51. SPS Customers API desplegada en Runtime Manager
Imagen 52. SPS Customers API status desde API Manager al estar corriendo en Cloudhub
Imagen 53. Test de llamada a endpoint /customers después de despliegue en Cloudhub con API Autodiscovery
Imagen 54. Opción Policies dentro de la API en API Manager
Imagen 55. Búsqueda de police Client ID Enforcement
Imagen 56. Aplicación de política de Client ID Enforcement
Imagen 57. Test de falla de llamada a endpoint /customers

Introducción y problemática

En SPS el equipo de marketing tiene la necesidad de tener disponible la información de nuestros clientes con la intención de poder generar campañas personalizadas que permitan mejorar la experiencia y la satisfacción del cliente, esta información se encuentra almacenada en una Base de Datos.

Para poder exponer esa información necesitamos desarrollar una API que se conecte a la base de datos para consultar dicha información.

1. La aplicación de MuleSoft que se construya debe contar con las siguientes características:
 - La información de los clientes debe estar expuesto en la siguiente ruta:
/api/v1/sps/customers
 - Dos archivos de propiedades, uno local y otro para dev.
 - Dos archivos uno para la implementación de la solución y otro donde se almacenen todos los elementos globales.
 - El usuario y contraseña de la base de datos debe estar protegida, haciendo uso secure properties.
2. Del ejercicio anterior se debe construir la especificación del API.
3. La aplicación debe estar desplegada en cloudfoundry.

Como un extra también puedes explorar mejorar la solución desarrollada con los siguientes tópicos:

1. Crear un API en API Manager.
2. La aplicación realizada en el ejercicio anterior, configurarla haciendo uso de API Autodiscovery.
3. Aplicar la política de client-id Enforcement para proteger nuestra API.

Acerca de la Base de Datos

El gestor de base de datos es MySQL y la consulta que realizaremos es la siguiente:

SELECT * FROM customers;

Estos son los valores de conexión de la base de datos:

Propiedad	Valor
Host	mudb.learn.mulesoft.com
Port	mule
User	mule
Database	training

Procedimiento

Comenzamos dentro de Anypoint Studio donde crearemos un nuevo **Mule Project** con el nombre **SPSCustomersAPI**.

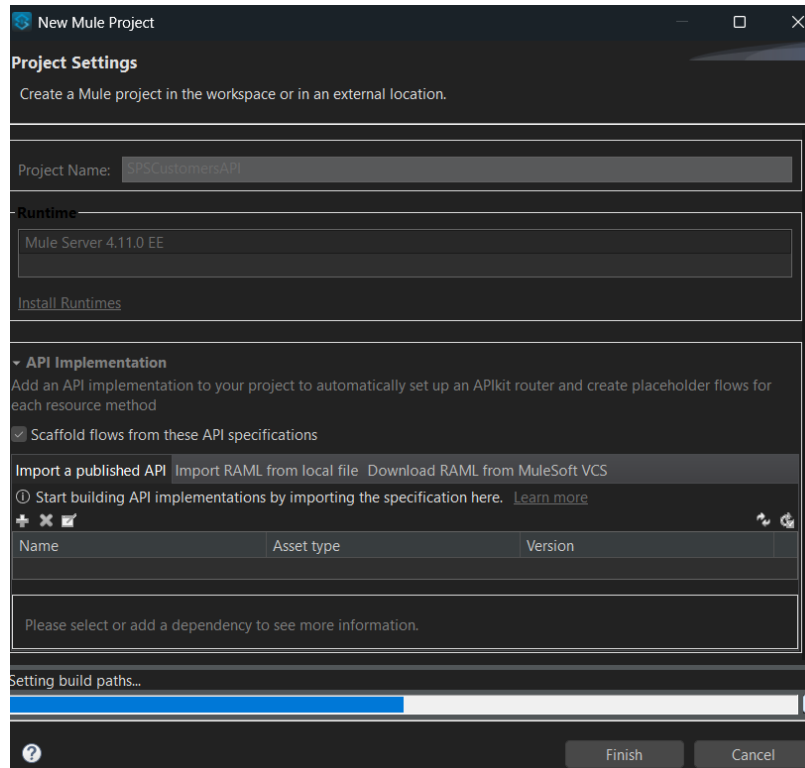


Imagen 1. Creación de un nuevo Mule Project

Para la separación de elementos globales hacemos clic derecho en **src/main/mule** y en **New > Mule Configuration File** para nombrarlo **global.xml**.

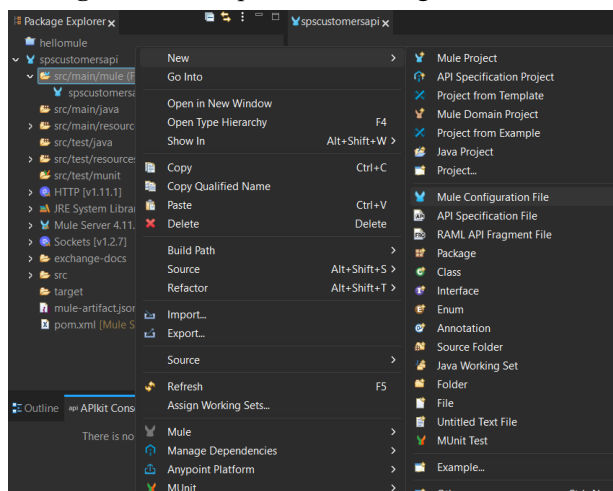


Imagen 3. Menú de src/main/mule

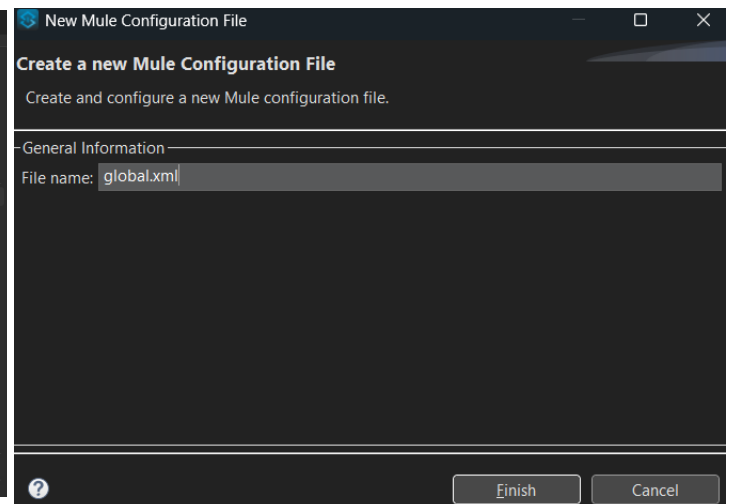


Imagen 2. New Mule Configuration File

Continuamos con la estructuración del proyecto y los archivos. En **src/main/resources** hacemos clic derecho > **New > File** y creamos los dos archivos de **local.properties** y **dev.properties**.

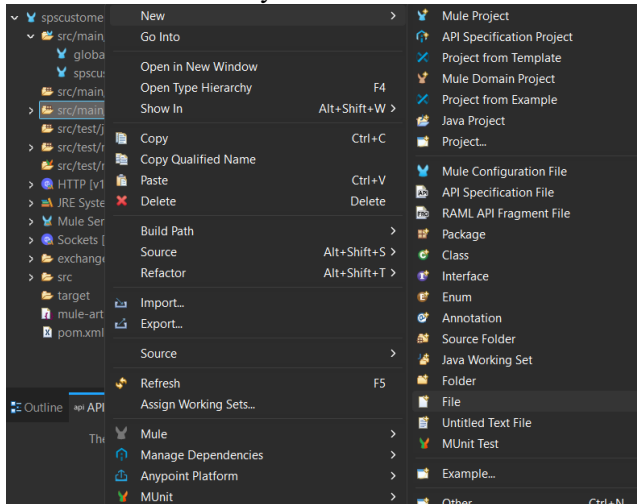


Imagen 5. Menú de creación de archivos de propiedades

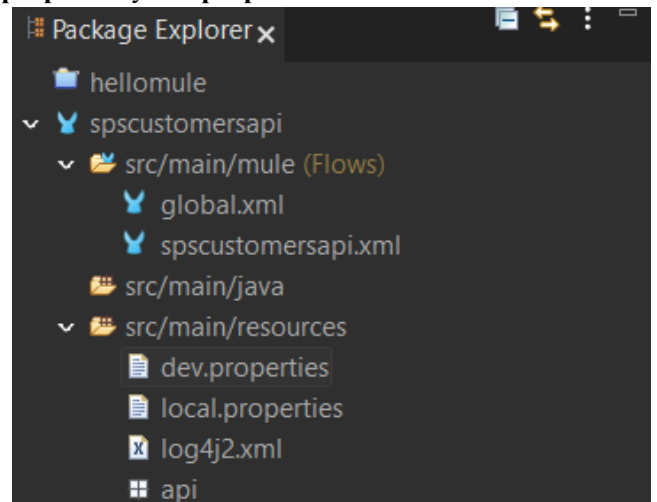


Imagen 4. Archivos de propiedades creados

Ahora volvemos al archivo base **spscustomersapi** y añadimos un **HTTP Listener** desde la **Mule Palette** añadimos un connector con los valores por defecto (por el momento) y en la ruta le ponemos **/api/v1/sps/customers**.

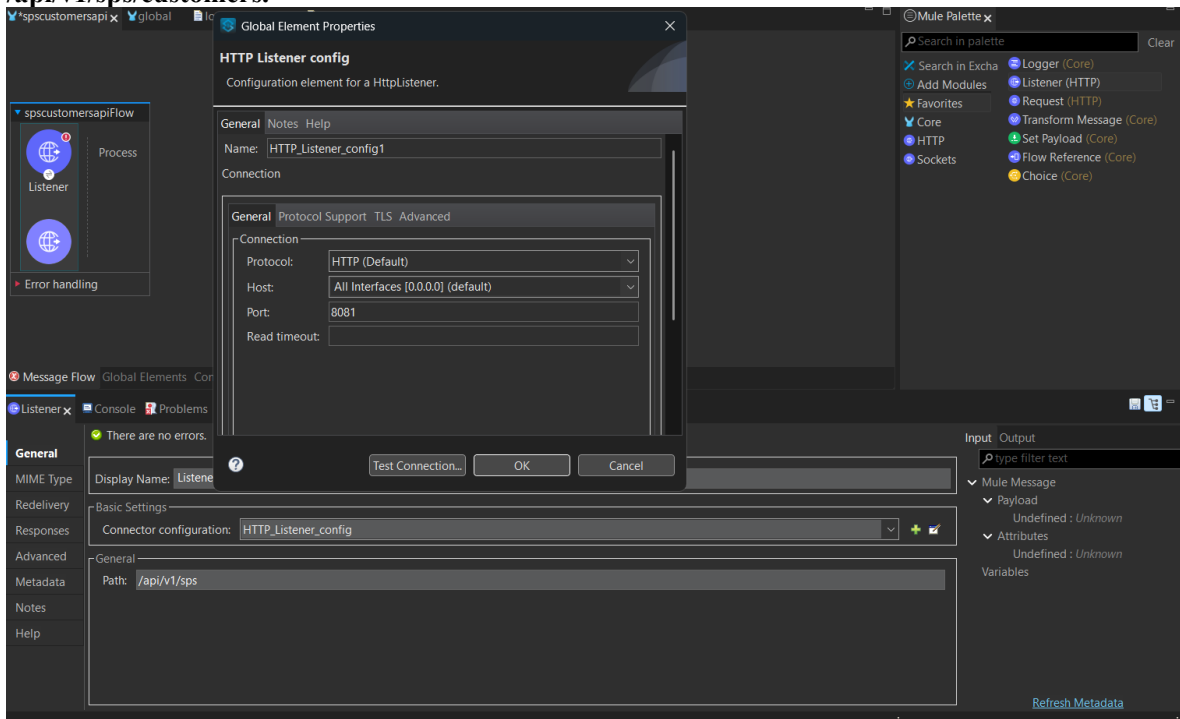


Imagen 6. HTTP Listener configuración

Ahora crearemos nuestra configuración para tener las propiedades de manera dinámica, es decir, que se extraigan desde los archivos que creamos de **local.properties** y **dev.properties**. Damos clic en la Tab de **Global Elements** y buscamos **Configuration Properties**

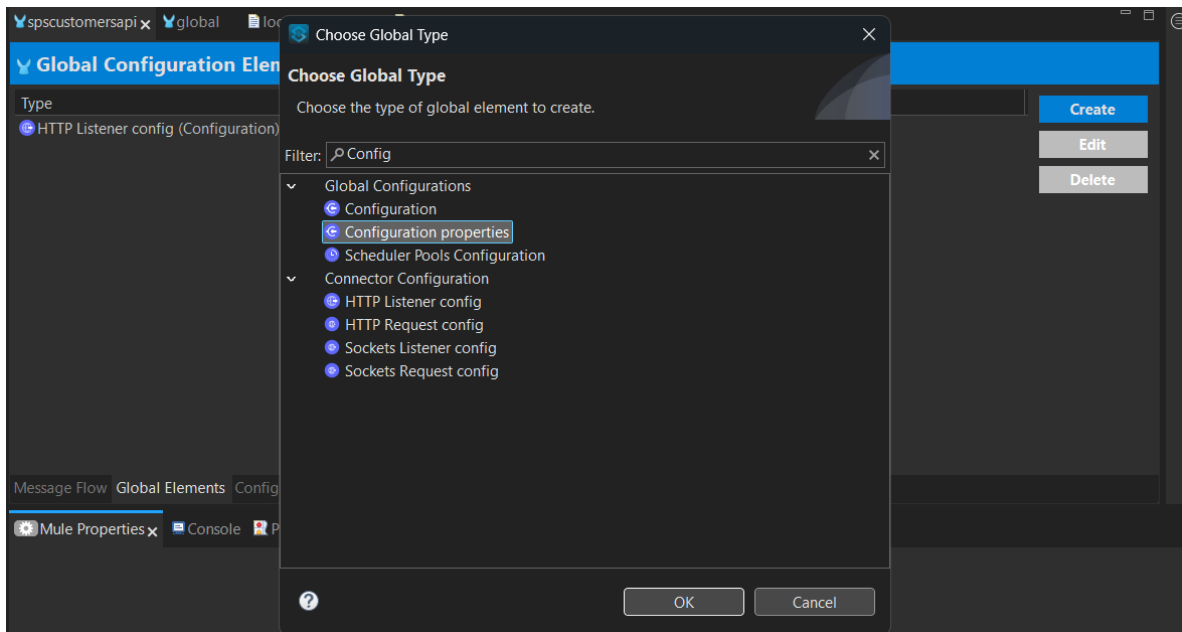


Imagen 7. Menú para crear *Configuration properties*

En la configuración de las *configuration properties* nos pedirá el archivo al que apuntará, para esto utilizamos la notación de variables que tenemos dentro de MuleSoft escribiendo **`${env}.properties`**.

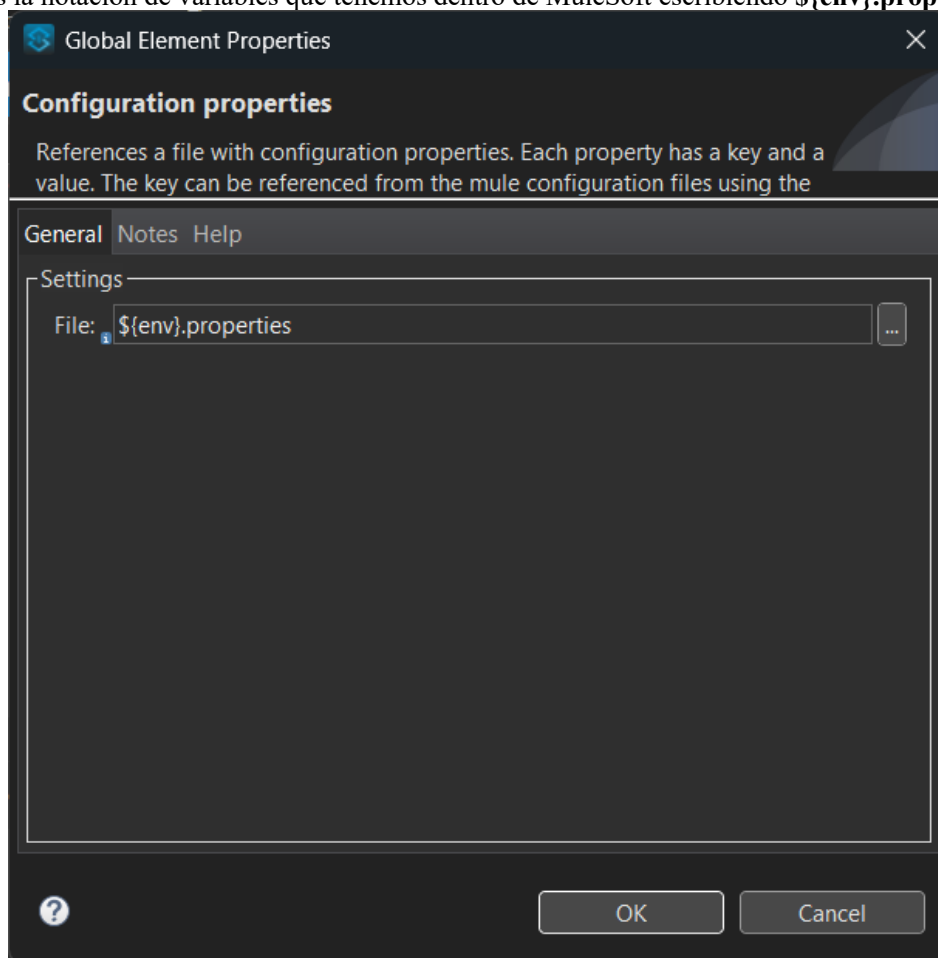


Imagen 8. Configuración del *Global Element Properties*

Aquí cometí un error y tendré que pasar la configuración de propiedades globales de la aplicación al archivo global que creamos. Esto dentro de Configuration XML y copiando las etiquetas correspondientes de una a otra.

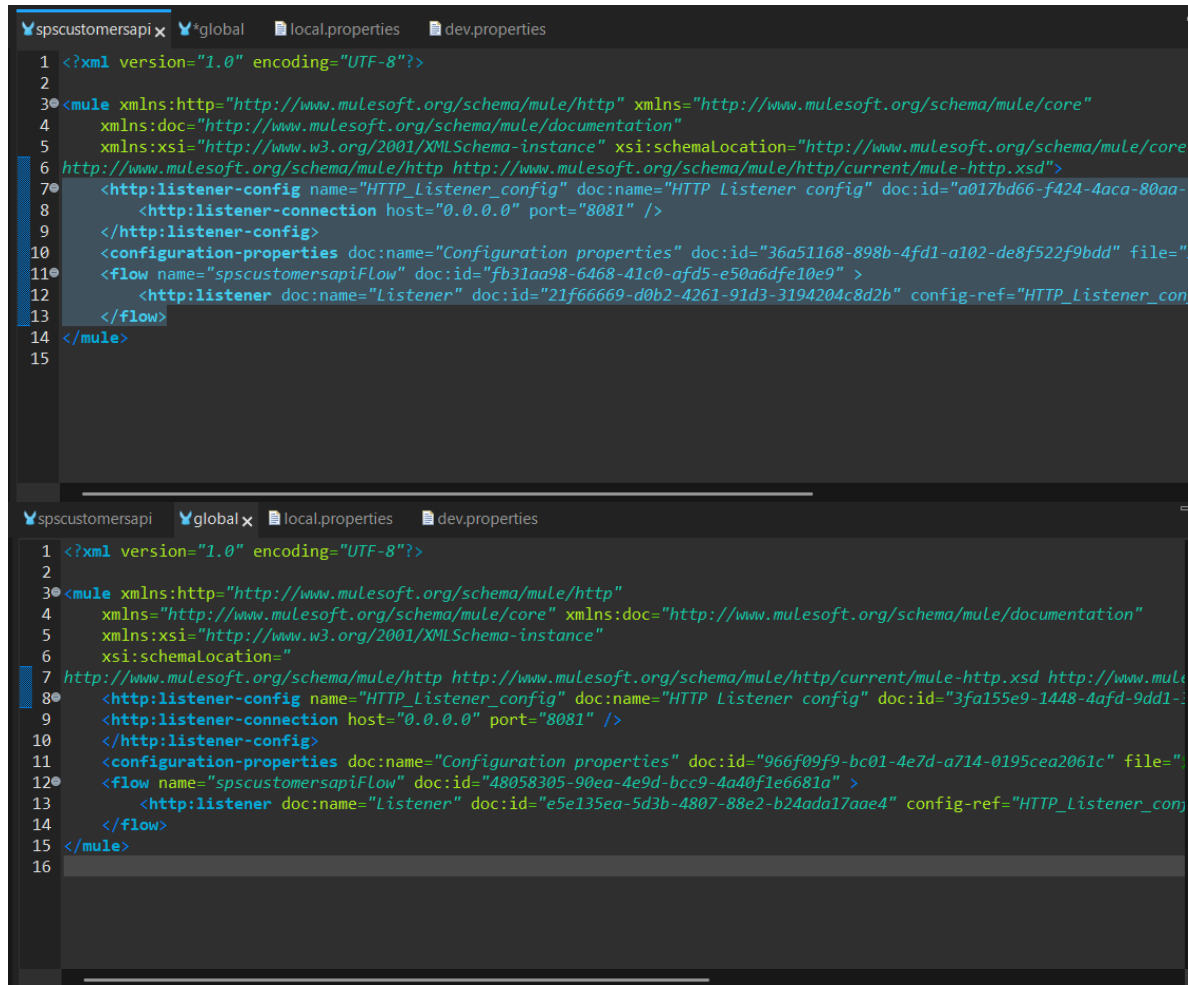


Imagen 9. HTTP Listener y Configuración Properties de spscustomersapi a global

Añadimos dentro de nuestros dos archivos de propiedades (local y dev) las siguientes líneas respectivamente para poder hacer la asignación dinámica de estas configuraciones.

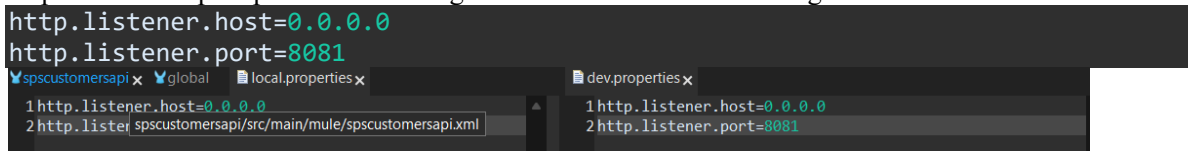


Imagen 10. Propiedades local y dev

Dentro de nuestro HTTP Listener asignamos los valores dinámicos dentro de la configuración con `${http.listener.port}` y `${http.listener.host}`.

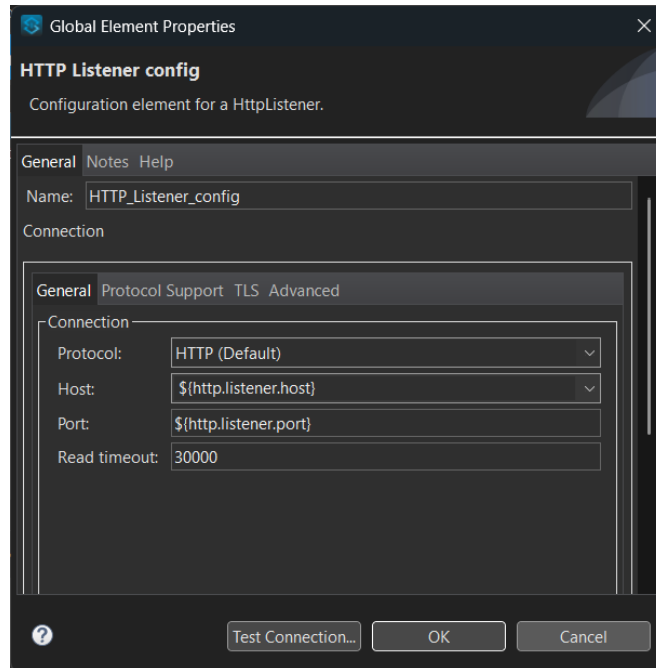


Imagen 11. Configuración dinámica de HTTP Listener

Creamos ahí mismo un nuevo elemento con **Create** y buscando **Global Property** para la variable del entorno en el que estamos, asignándole la siguiente configuración:

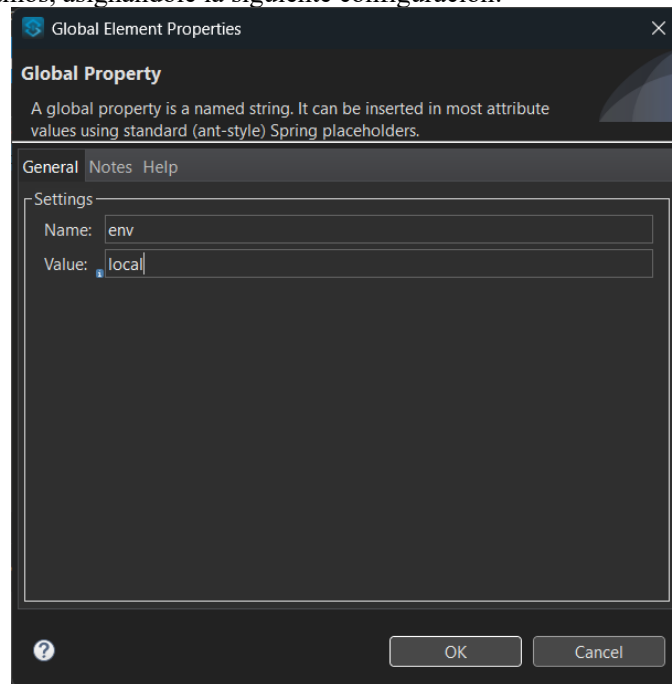


Imagen 12. Configuración de Global Property

Ahora crearemos los archivos de propiedades **secure** de igual forma que los de **dev.properties** y **local.properties** los cuales nombraremos **dev.secure.properties** y **local.secure.properties**.

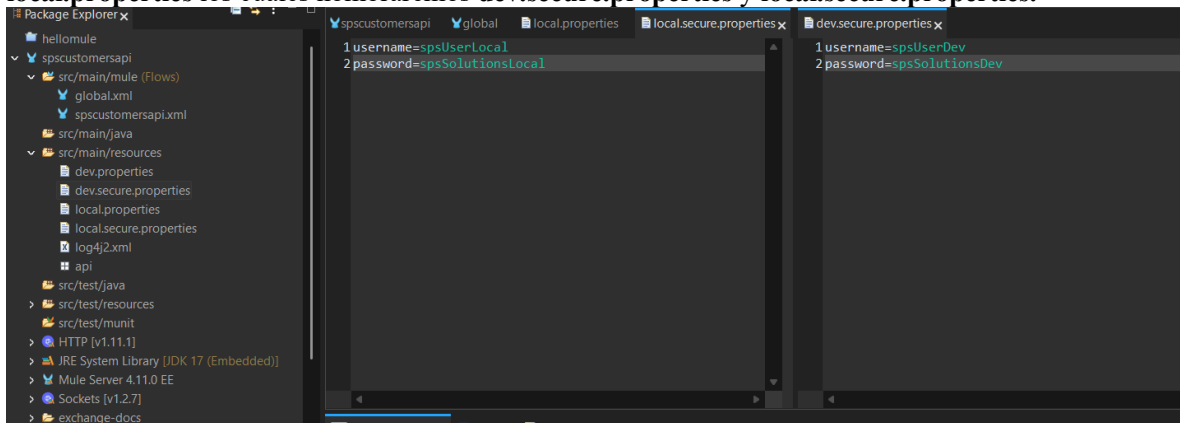


Imagen 13. Archivos dev/local.secure.properties

Añadimos el módulo de seguridad **Mule Secure Configuration Properties** haciendo clic en **global.xml** dentro de la Tab de **Message Flow** en la **Palette > Search in Exchange** buscamos el módulo y damos clic en **Add**.

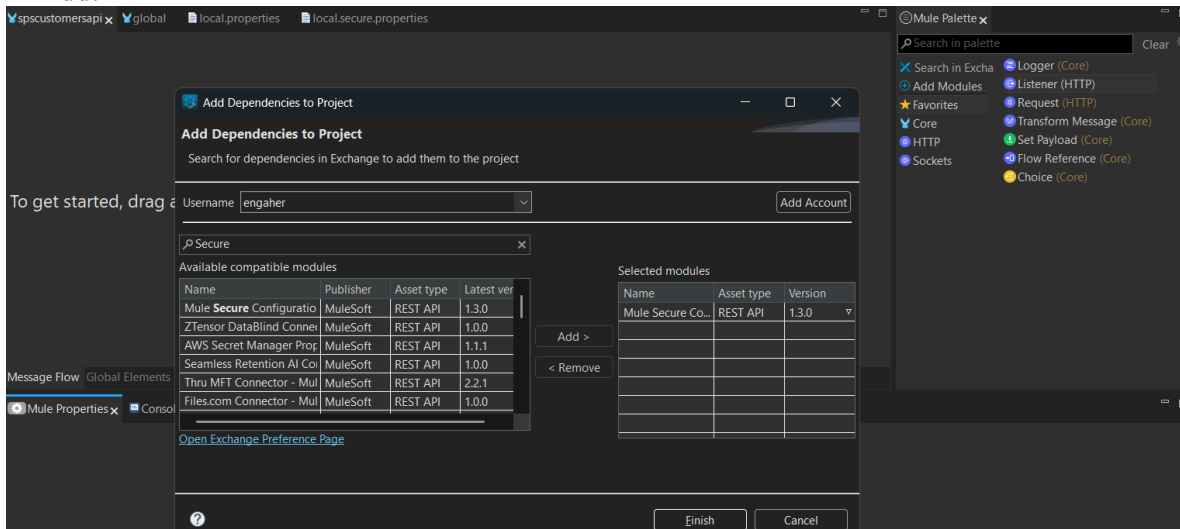


Imagen 14. Importación del módulo Mule Secure Configuration Properties

Ahora que tenemos el módulo añadido vamos a la vista de **Global Elements** dentro de **global.xml** y damos clic en **Create** y buscamos **Secure Properties Config**. Dentro escribimos **\${env}.secure.properties** para añadir y apuntar al archivo de manera dinámica. Elegimos la Key con el algoritmo **Blowfish** y escribimos **\${secure.key}** (esta variable la asignaremos más tarde).

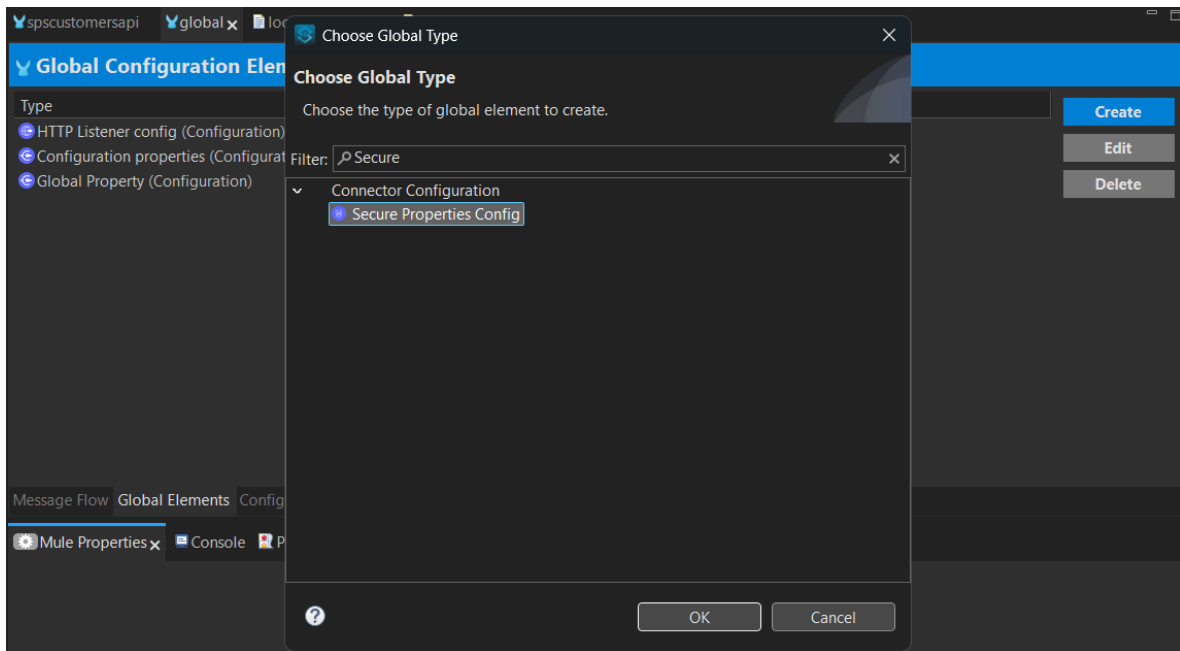


Imagen 15. Menú para añadir Secure Properties Config

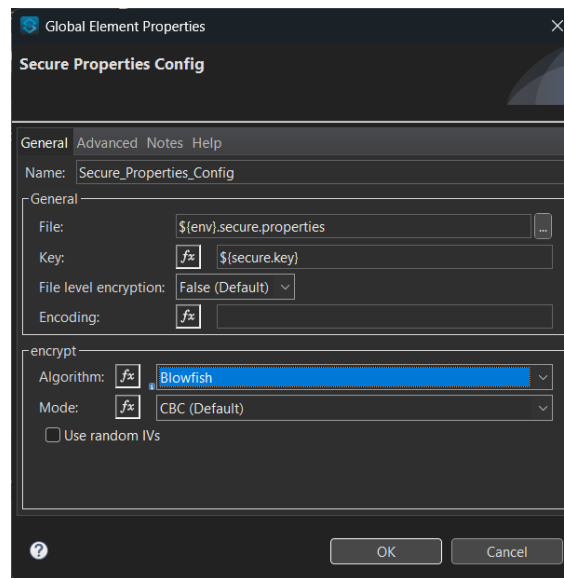


Imagen 16. Configuración del Secure Properties Config

Escribiremos la configuración de acceso a la base de datos dentro de nuestro archivos de configuración. Pero primero necesitamos cifrar la contraseña de acceso a la base de datos por seguridad. Para ello utilizaremos el **Secure Properties Tool Jar** desde la línea de comandos para cifrar la contraseña de acceso a la base de datos corriendo lo siguiente en una terminal apuntando al directorio donde está nuestro archivo **secure-properties-tool.jar**:

```
java -cp secure-properties-tool-j17.jar com.mulesoft.tools.SecurePropertiesTool string encrypt Blowfish CBC MyMuleSoftKey "mule"
```

```
PS C:\Users\kikga\Downloads> java -cp secure-properties-tool-j17.jar com.mulesoft.tools.SecurePropertiesTool string encrypt Blowfish CBC MyMuleSoftKey "mule"
hn5PdGLGVPw=
PS C:\Users\kikga\Downloads> |
```

Imagen 17. Cifrado de contraseña de acceso a la base de datos

Colocamos nuestras credenciales de acceso a la base de datos en **local.properties** y **dev.properties** quedando de la siguiente manera:

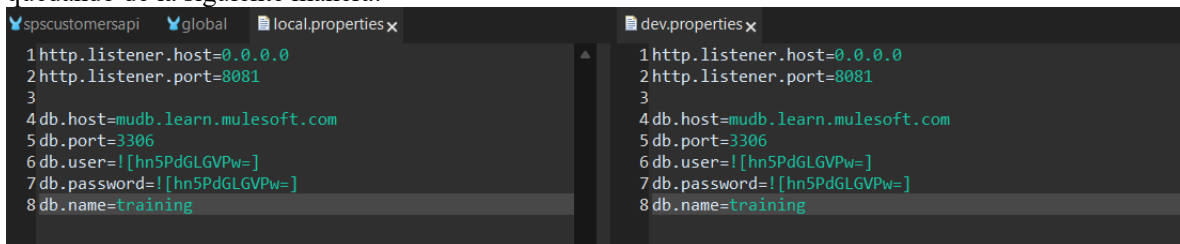


Imagen 18. Variables de acceso a la base de datos

En el paso anterior cometí un error, el **db.user** y **db.password** deben ir en el **secure.properties**. Ahora dentro de **global.xml** buscaré en **Exchange** una dependencia para poder crear la conexión con la base de datos de MySQL llamada **Database Connector**.

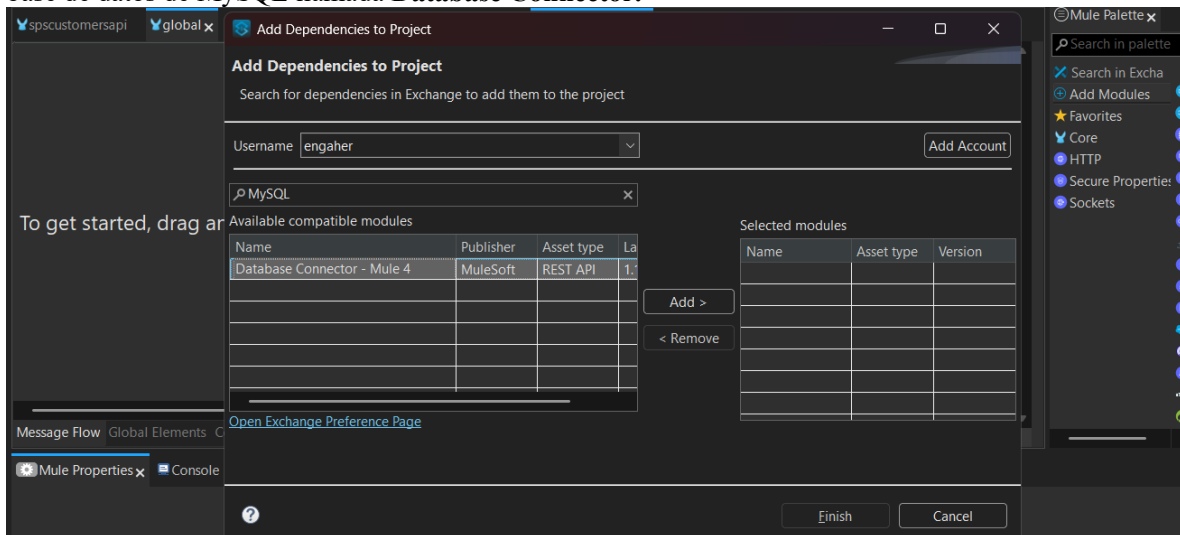


Imagen 19. Dependencia Database Connector en Exchange

Después de agregar este módulo podremos crear en nuestro **global.xml** un nuevo Global para **Database Config** buscándolo en **Create>Database Config**.

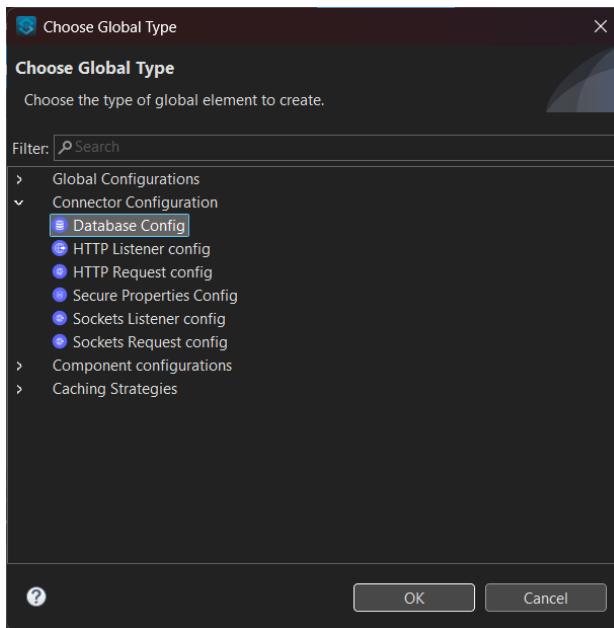


Imagen 20. Database Config en Global Elements de global.xml

Seleccionamos en **Connection** el tipo correspondiente a **MySQL** y añadimos (configuramos) el controlador requerido para la conexión.

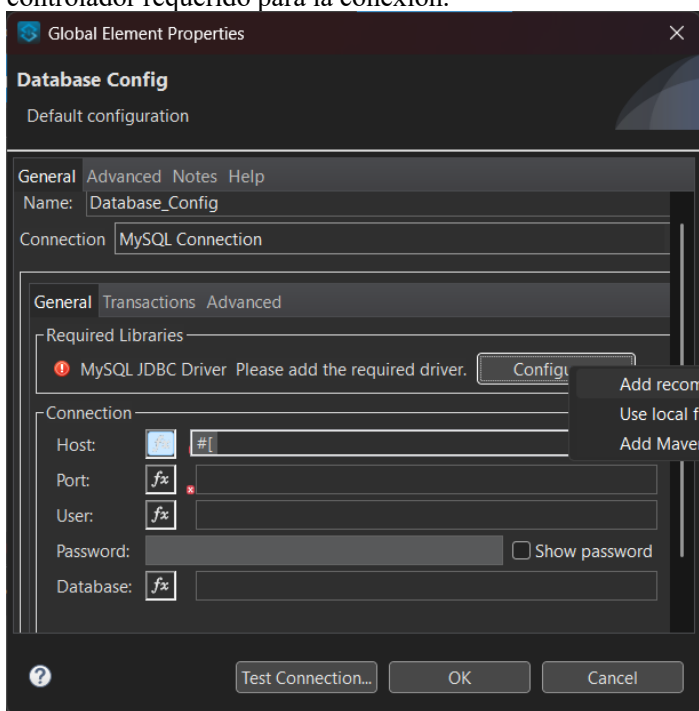


Imagen 21. Database Config añadiendo controlador para la conexión

Llenamos los campos de **Host**, **Port**, **User**, **Password** y **Database** haciendo referencia a las variables de nuestro `.properties` y `.secure.properties` de la siguiente manera:

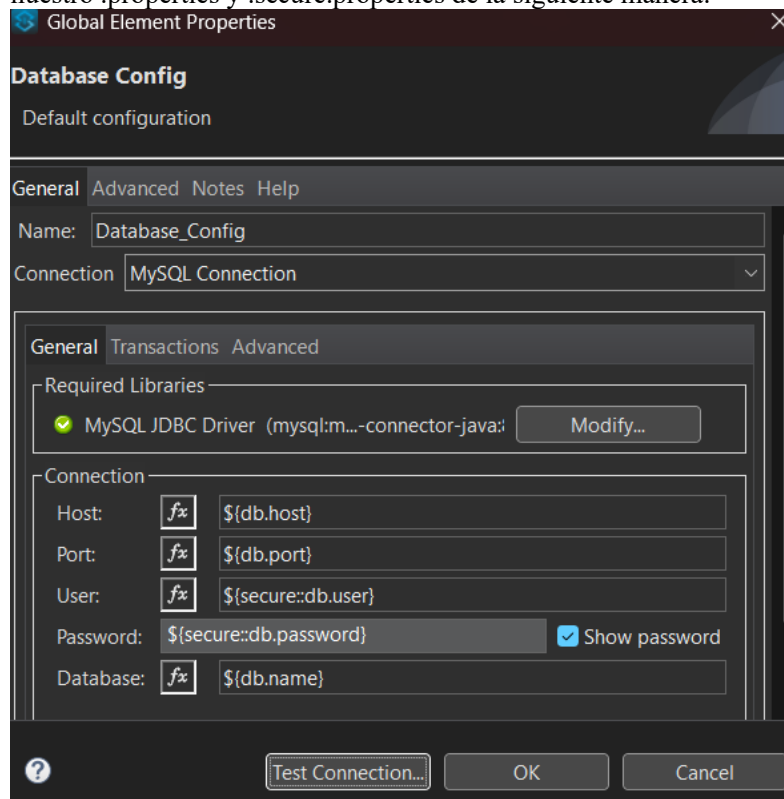


Imagen 22. Configuración de la conexión a base de datos

Probamos la conexión dando clic en **Test Connection...** pero antes debemos quitar el `${secure.key}` que pusimos en nuestras **Secure Properties** poniendo **MyMuleSoftKey** provisionalmente SÓLO para probar la conexión:

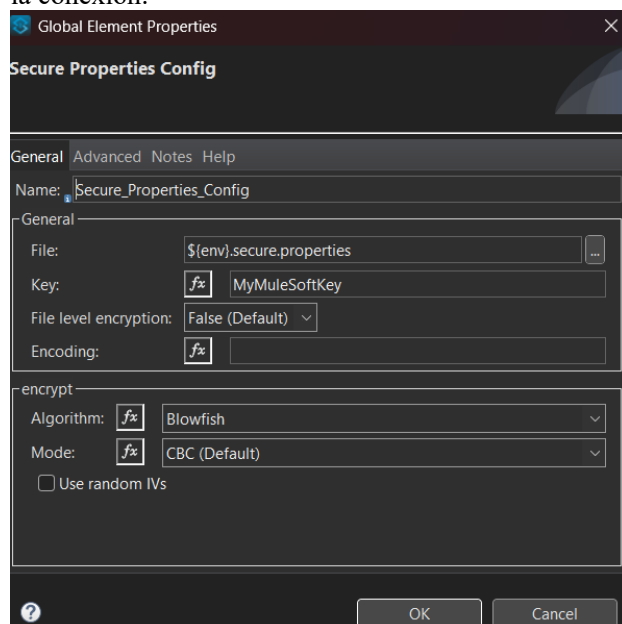


Imagen 23. Cambio provisional de secure.key

Probamos conexión y verificamos que sea exitosa:

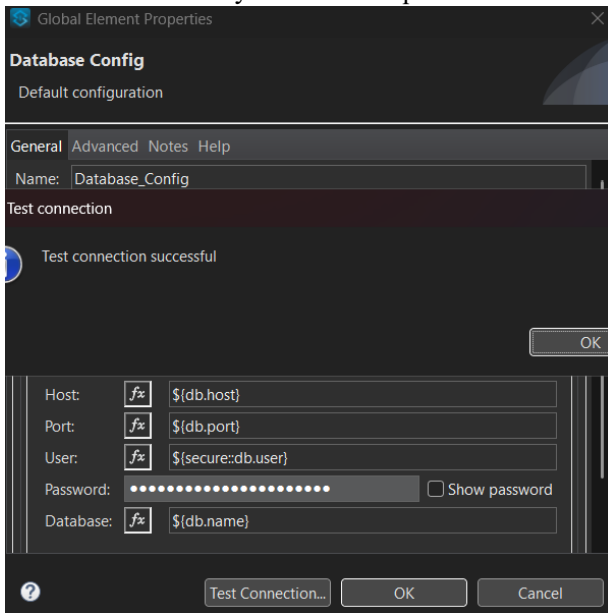


Imagen 24. Test de conexión a la base de datos

Ahora pasamos a la implementación del **SELECT * FROM customers** para ello iremos a nuestro archivo `spscustomersapi.xml` y buscaremos en la paleta **Database > Select** y lo añadiremos dentro del Flow.

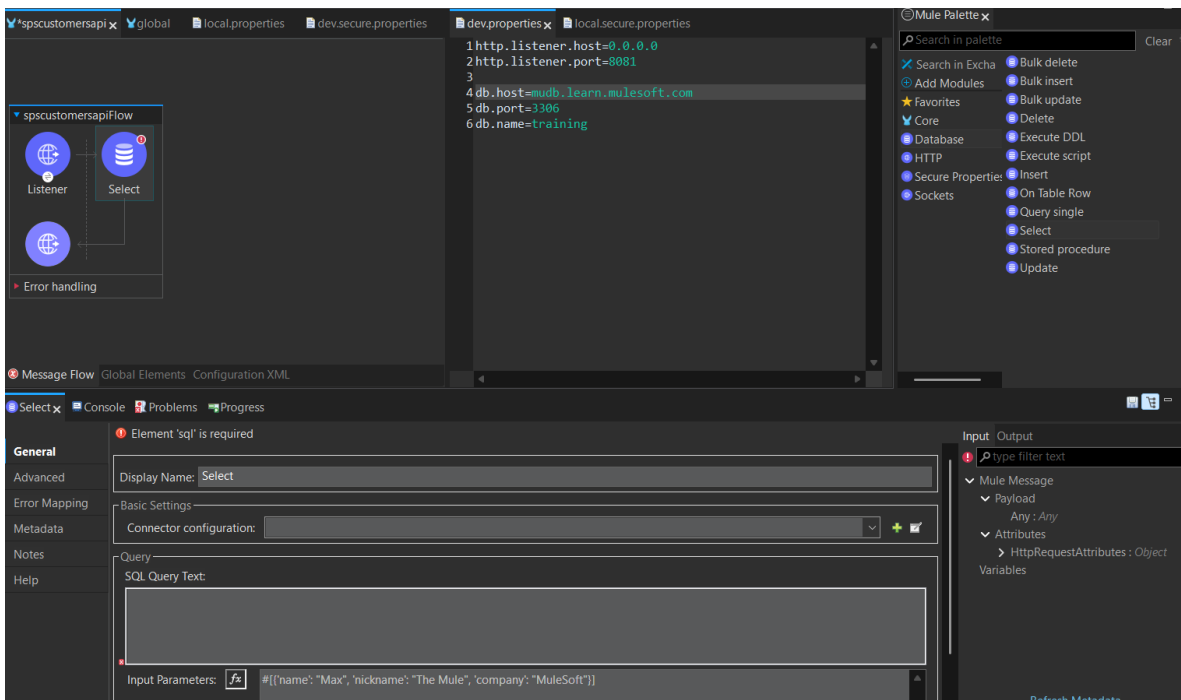


Imagen 25. Implementación de SELECT dentro del Flow principal

En la configuración del **Select** seleccionamos la configuración de base de datos que creamos anteriormente y ponemos la consulta **SELECT * FROM customers** en **SQL Query Text**.



Imagen 26. Configuración de componente Select

Aquí utilicé un componente que vi en la paleta, ya que este Select nos entregaría los resultados de una forma que no sabemos, por lo que para estar seguros que lo retorne en formato JSON arrastramos un **Transform Message** de la **Palette** al Flow y colocamos el siguiente código en su configuración:

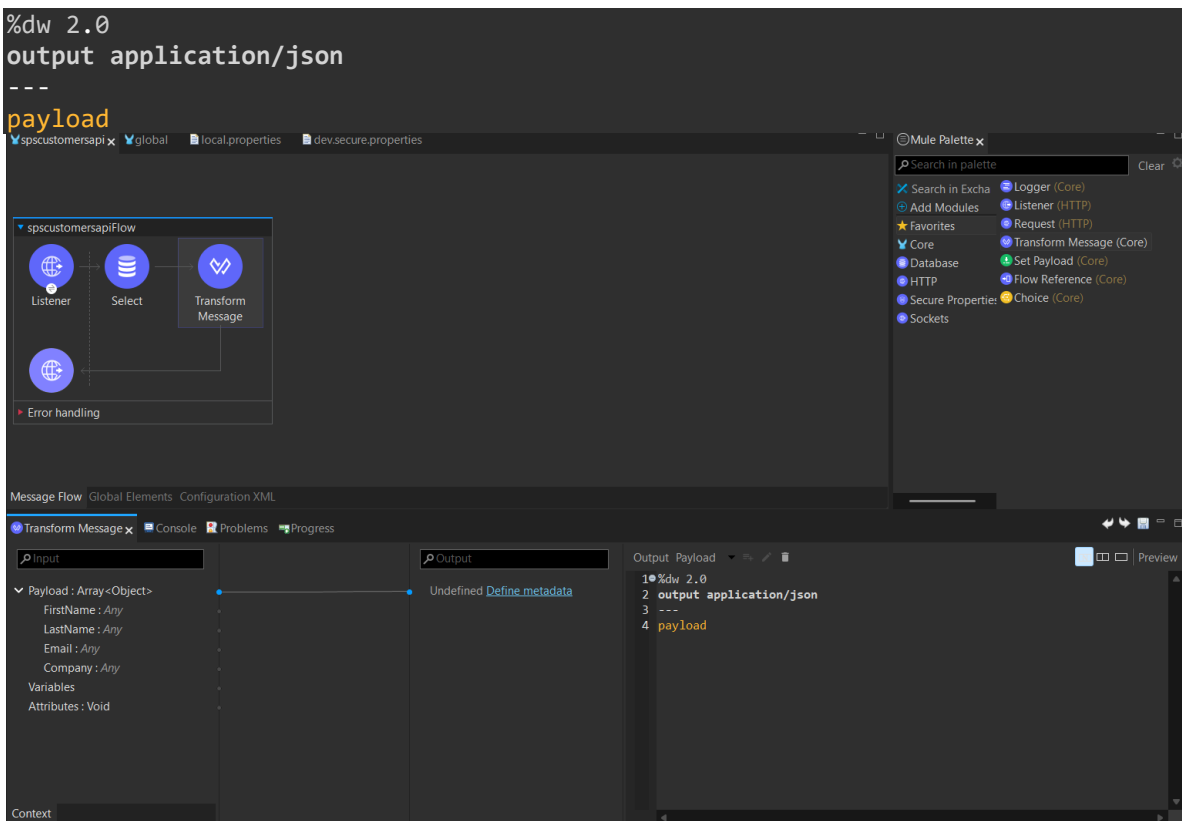


Imagen 27. Componente Transform Message agregado al Flow

Ahora bien, regresamos la configuración de **Secure Properties Config** tal como estaba con el `#{secure.key}`:

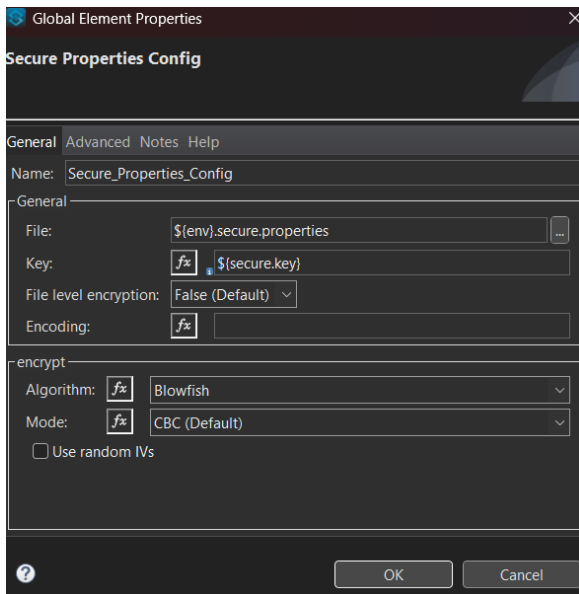


Imagen 28. Configuración reestablecida de Secure Properties

Y probaremos corriendo la aplicación en local con **Clic derecho en el proyecto > Run as > Run Configurations...** y nos aseguramos que nuestro proyecto **spscustomersapi** esté seleccionado y que tengamos estas dos variables de entorno en la pestaña de **Environments**. En caso de no tenerlas las podemos añadir haciendo clic en **Add...**

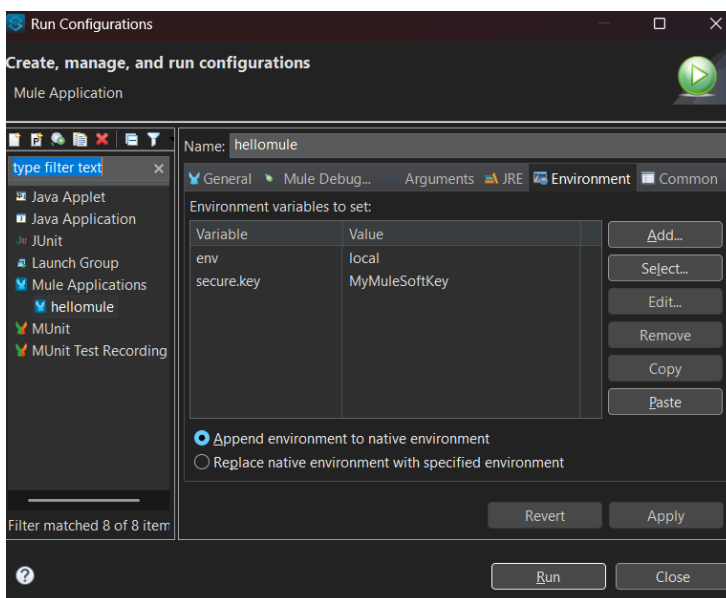


Imagen 29. Run Configurations para probar localmente

Una vez que nuestra aplicación este corriendo saldrá **DEPLOYED** en la consola y podremos probar con nuestro **REST CLIENT** (en mi caso PostMan) el endpoint, haciendo la llamada GET a <http://localhost:8081/api/v1/sps/customers>

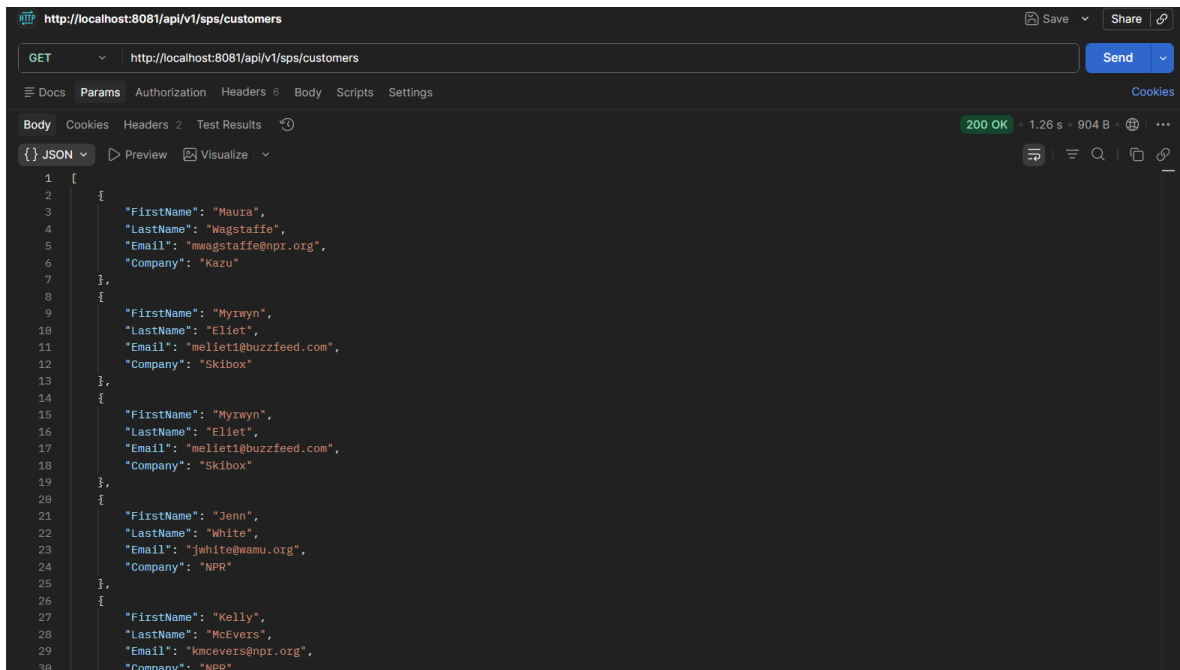


Imagen 30. Test de llamada a la API antes de despliegue a Cloudhub

¡Listo! Nuestra aplicación funciona localmente y podemos ver los clientes. Ahora continuaremos desplegando a Cloudhub **haciendo clic derecho en el proyecto > Anypoint Platform > Deploy to Cloudhub**. Aquí llenamos en la pestaña de **Properties** los secure.key y el env.

Otra alternativa que encontré ya que a mi no me funciona el despliegue a Cloudhub desde la plataforma de escritorio. Es **Clic derecho en el proyecto > Export > Mule > Anypoint studio Project to Mule Deployable Archive**. Y subiendo el archivo directamente a la web.

Deploy Application

Application Name

spscustomersapi

Deployment Target

Shared Space US East (Ohio)

CloudHub 2.0

Application File

No file has been loaded

Choose file

[This is a](#)

[public deployment target.](#)

Import file from Exchange

Upload file

Runtime

Ingress

Properties

Monitoring

Firewall Rules

Table view

Text view

Use properties to change the way your app behaves. [Learn more](#)

secure.key

MyMuleSoftKey

Protect

...

env

dev

Protect

...

New Key

New Value

Deploy Application

Application Name

spscustomersapi

Deployment Target

Shared Space US East (Ohio)

CloudHub 2.0

Application File

spscustomersapi.jar

Choose file

[This is a](#)

[public deployment target.](#)

Runtime

Ingress

Properties

Monitoring

Firewall Rules

Table view

Text view

Use properties to change the way your app behaves. [Learn more](#)

secure.key

MyMuleSoftKey

Protect

...

env

dev

Protect

...

New Key

New Value

Deploy Application

Imagen 31. Despliegue a Cloudhub desde la web seleccionando archivo

● spscustomersapi

Stop

...

...

✓ Desired configuration applied successfully. 1 / 1 Replicas started

✕

Application status: ● Running

Configuration: 3efe32

Last updated: 2026-05-10 1:24:06PM

Replicas: 1 / 1 started

Public Endpoint: <https://spscustomersapi-m20wgk.5sc6y6-1.usa-e2.cloudhub.io>

Target name: Cloudhub-US-East-2

Target type: Shared Space

Configuration 3efe32

Applying changes will create a new configuration for your application

Apply Changes

Application File

spscustomersapi.jar

Choose file

Version 1.0.0

Deployment Target

Ingress

Properties

Monitoring

Firewall Rules

Imagen 32. Despliegue exitoso a Cloudhub.

Ahora podemos probar a llamar a GET /api/v1/sps/customers con el public endpoint que nos proporciona Cloudfoundry.

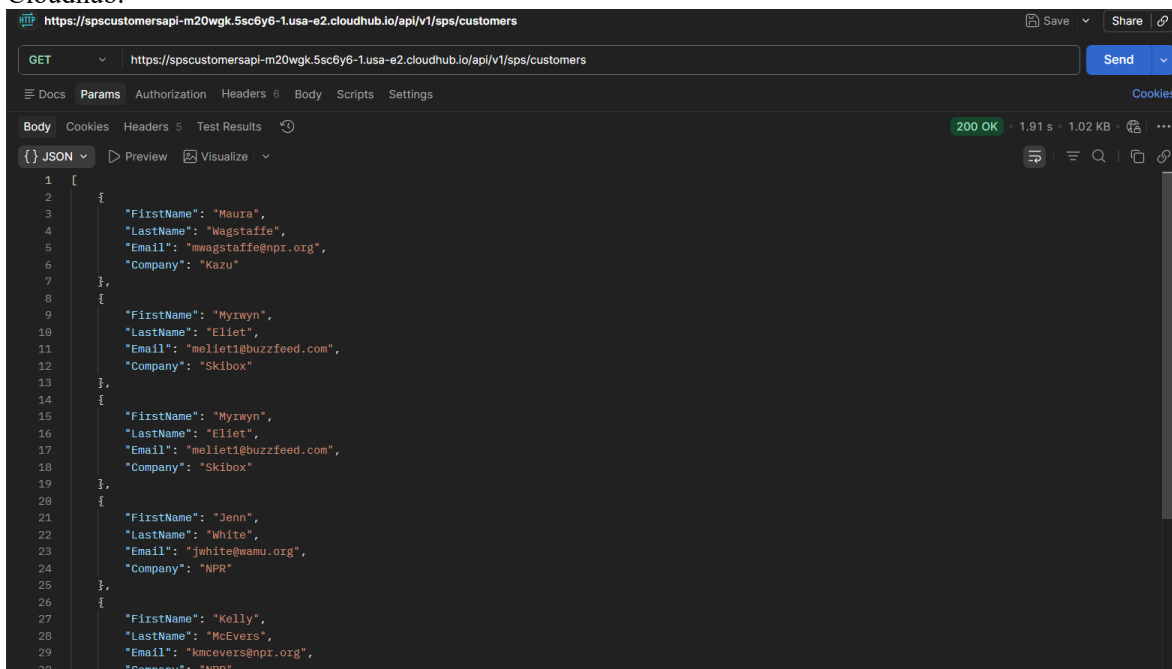


Imagen 33. Llamada GET al endpoint de customers exitosa

Para crear la especificación nos vamos a [Anypoint Platform](#) > APIs > Design Center > Create > New API Specification. Colocamos el nombre SPS Customers API y seleccionamos el Guide me through it

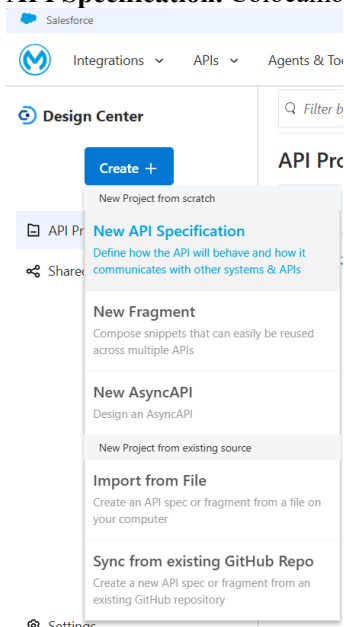


Imagen 35. Menú de crear una especificación nueva.

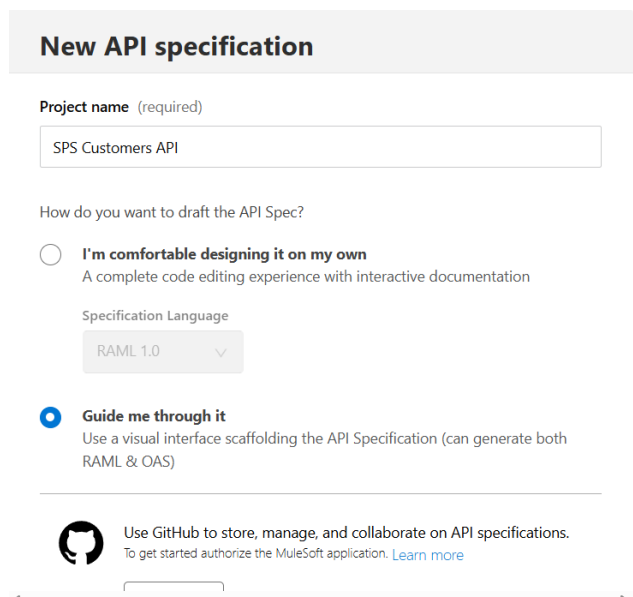
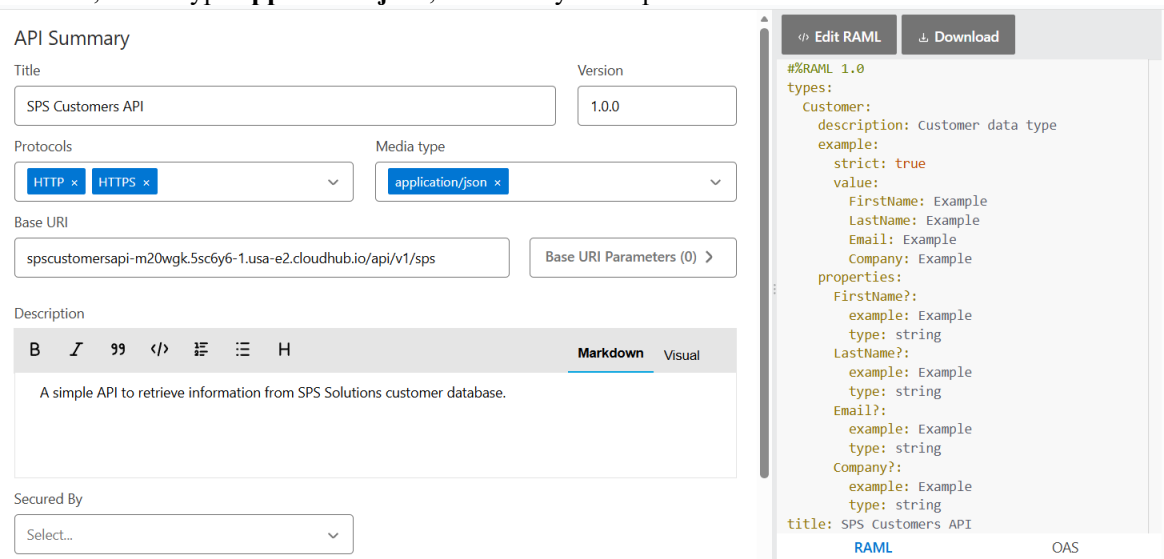


Imagen 34. Nombre de API Specification

Ya en el **API Designer** colocamos el título **SPS Customers API**, versión **1.0.0**, protocolos **HTTP** y **HTTPS**, media type **application/json**, base URI y descripción:



API Summary

Title: SPS Customers API Version: 1.0.0

Protocols: HTTP x HTTPS x Media type: application/json x

Base URI: spscustomersapi-m20wgk5sc6y6-1.usa-e2.cloudhub.io/api/v1/sps Base URI Parameters (0) >

Description: A simple API to retrieve information from SPS Solutions customer database.

Secured By: Select...

API Summary

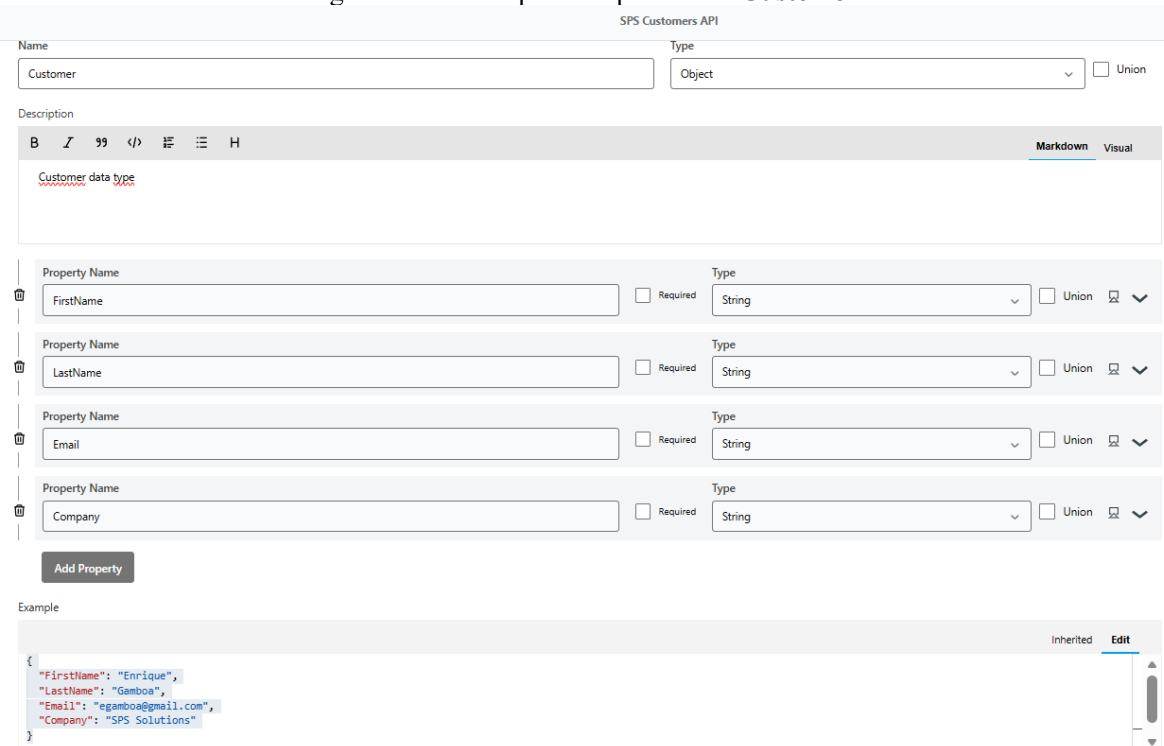
```

#%RAML 1.0
types:
  Customer:
    description: Customer data type
    example:
      strict: true
      value:
        FirstName: Example
        LastName: Example
        Email: Example
        Company: Example
    properties:
      FirstName?:
        example: Example
        type: string
      LastName?:
        example: Example
        type: string
      Email?:
        example: Example
        type: string
      Company?:
        example: Example
        type: string
    title: SPS Customers API
  
```

Imagen 36. API Designer lleno

Ahora añadimos un nuevo **Data type** seleccionando el ícono + junto a la pestaña de **DataTypes** a la izquierda.

Llenamos los datos con los siguientes valores para el tipo de dato **Customer**.



SPS Customers API

Name: Customer Type: Object Union

Description: Customer data type

Property Name: FirstName Type: String Required: Union

Property Name: LastName Type: String Required: Union

Property Name: Email Type: String Required: Union

Property Name: Company Type: String Required: Union

Add Property

Example: { "FirstName": "Enrique", "LastName": "Gamboa", "Email": "egamboa@gmail.com", "Company": "SPS Solutions" }

Imagen 37. Tipo de dato Customer lleno

Finalmente copiamos el siguiente ejemplo para pegarlo en el campo **Example** al final de la página. Hacemos click en el botón **Edit** para poder modificarlo

```
{
```

```

"FirstName": "Enrique",
"LastName": "Gamboa",
"Email": "egamboa@gmail.com",
"Company": "SPS Solutions"
}

```

Continuamos añadiendo un recurso dando clic en el ícono + en **Resources** para **Add Resources**. Configuramos el siguiente recurso para **/customers**.

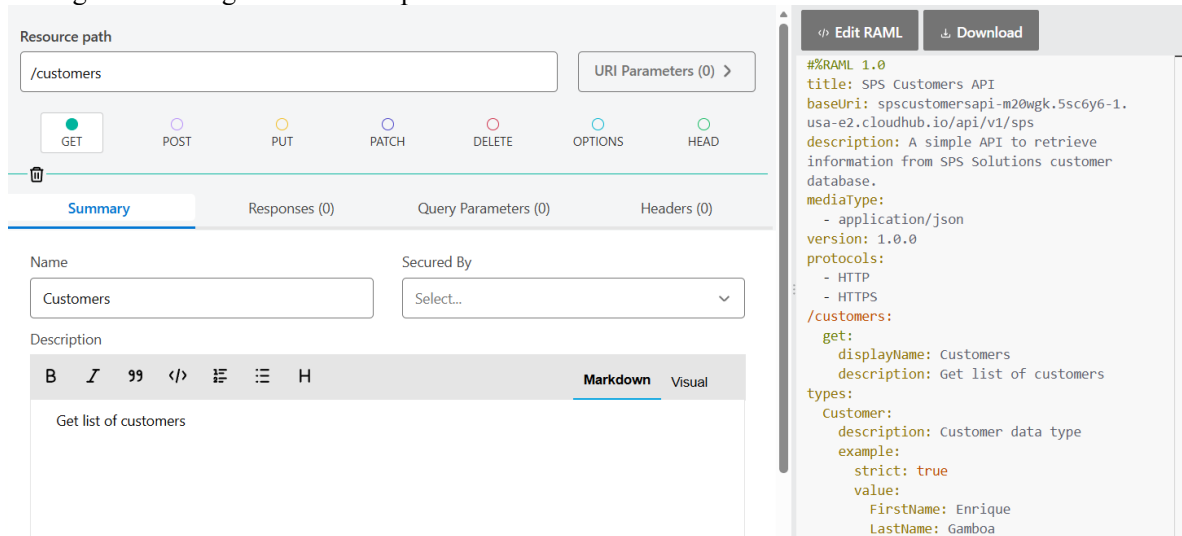


Imagen 38. Creación del recurso `/customers`

Seleccionamos la pestaña de **Responses** y hacemos clic en **Add new response**, nos aseguramos que el código de estatus sea **200 OK** y hacemos clic en **Add body**. Nos aseguramos que el media type sea **application/json** y el tipo sea **Array**. Expandimos el cuerpo con la flecha de la derecha y seleccionamos el tipo de dato **Customer** que acabamos de crear.

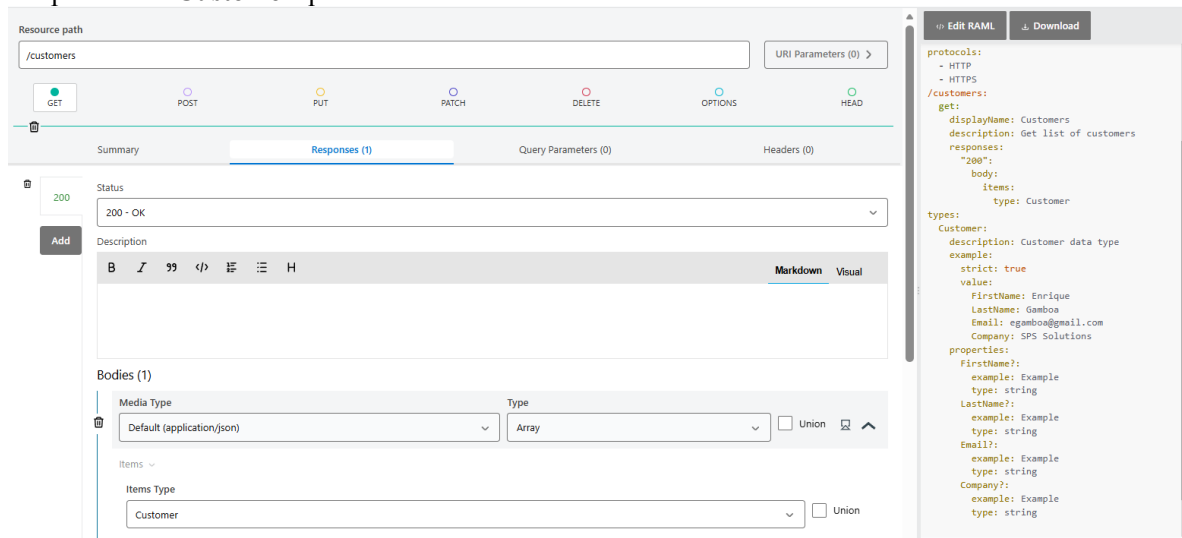


Imagen 39. Recurso `/customers` con la configuración llena

Ahora lo probaremos la API para ello seleccionamos del panel derecho el ícono de la flecha llamado **Try it** y seleccionamos en server el **Mocking Service**. Verificamos que funcioné dándonos el ejemplo que colocamos.

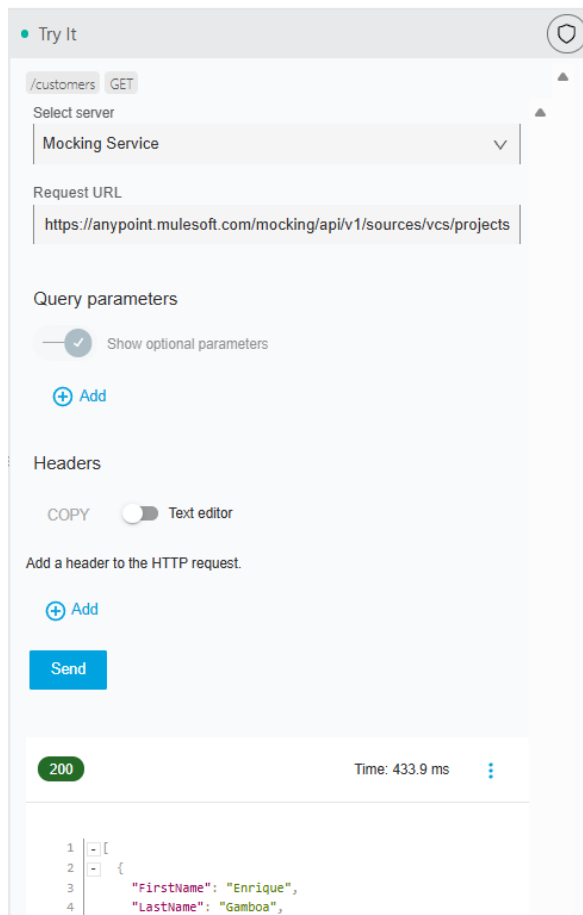


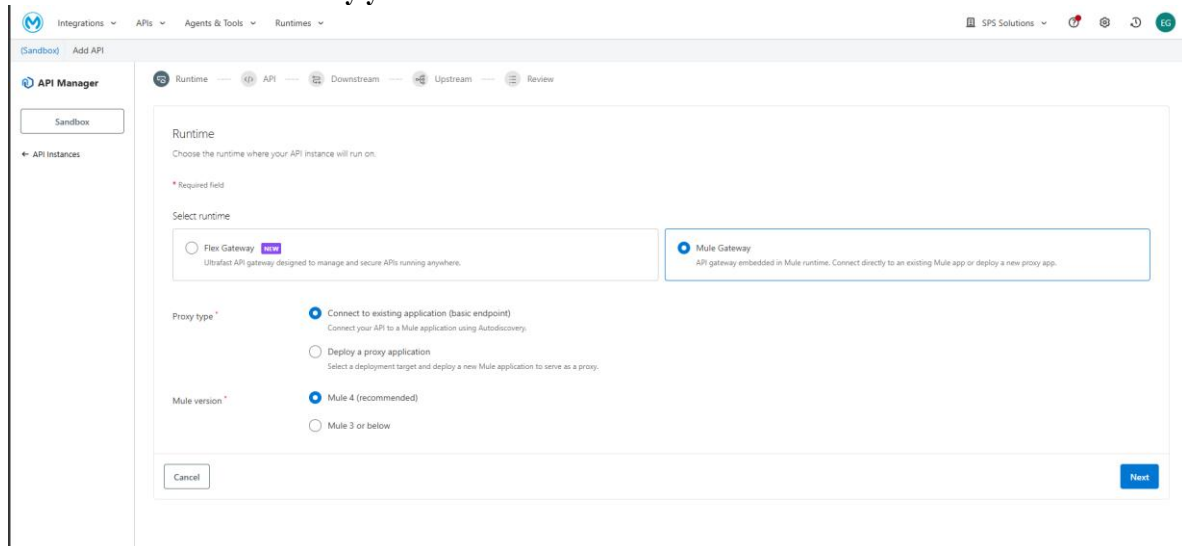
Imagen 40. Test con Mocking Service

Ahora que lo hemos probado procedemos a publicarlo en **Exchange** dando clic en **Publish** (arriba a la derecha) > **Publish to Exchange**.

Extras

APIManager

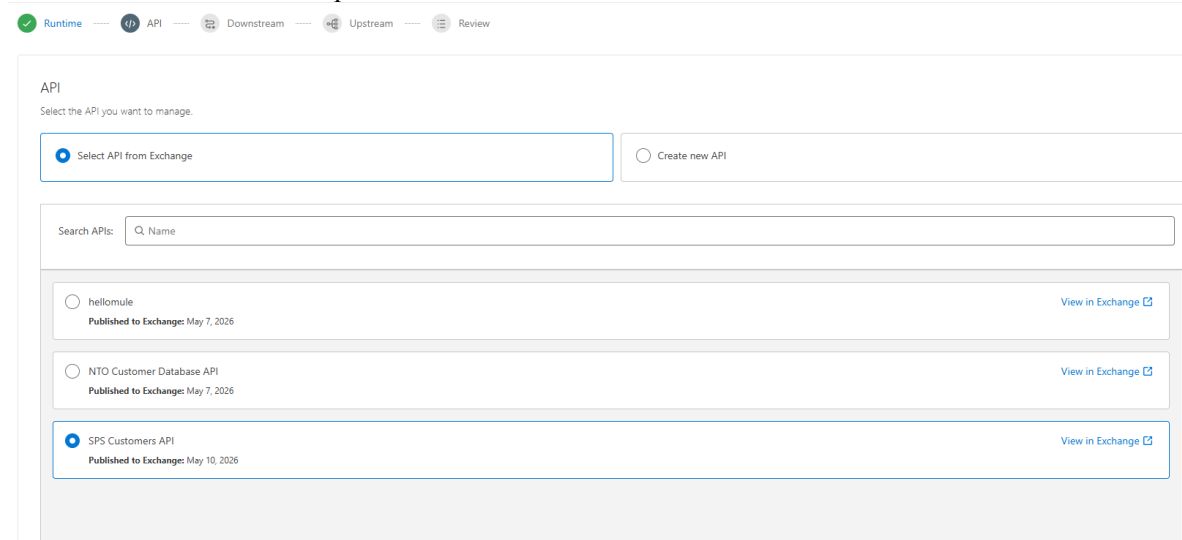
Dentro de [Anypoint Platform](#) accedemos al **API Manager** y hacemos clic en **Add > Add New API**. Seleccionamos **Mule Gateway** y **Next**.



The screenshot shows the 'Add New API' wizard in the API Manager, specifically the 'Runtime' step. The left sidebar shows 'API Manager' and 'Sandbox'. The main area has tabs for 'Runtime', 'API', 'Downstream', 'Upstream', and 'Review'. The 'Runtime' tab is active, showing a form to choose the runtime where the API instance will run. The form includes a 'Select runtime' section with two options: 'Flex Gateway' (disabled) and 'Mule Gateway' (selected). Below this is a 'Proxy type' section with two options: 'Connect to existing application (basic endpoint)' (selected) and 'Deploy a proxy application'. At the bottom is a 'Mule version' section with two options: 'Mule 4 (recommended)' (selected) and 'Mule 3 or below'. A 'Cancel' button is at the bottom left and a 'Next' button is at the bottom right.

Imagen 41. Creación de nueva API dentro de API Manager

En la siguiente pantalla escogeremos la opción **Select API from Exchange** para aprovechar la especificación de API que hicimos anteriormente durante el procedimiento no extra. Así que seleccionamos nuestra API Specification llamada **SPS Customers API**.



The screenshot shows the 'API' step of the wizard. It has tabs for 'Runtime', 'API', 'Downstream', 'Upstream', and 'Review'. The 'API' tab is active, showing a form to select the API you want to manage. There are two options: 'Select API from Exchange' (selected) and 'Create new API'. Below this is a search bar labeled 'Search APIs:' with a search icon and the text 'Q Name'. Below the search bar is a list of APIs. The list contains three items: 'hellomule' (Published to Exchange: May 7, 2026), 'NTO Customer Database API' (Published to Exchange: May 7, 2026), and 'SPS Customers API' (Published to Exchange: May 10, 2026). The 'SPS Customers API' item is selected. Each item has a 'View in Exchange' link.

Imagen 42. Selección de API Specification realizada anteriormente

Damos clic en **Next** sin cambiar nada, al final en **Review** debería verse algo así:

✓ Runtime — ✓ API — ✓ Downstream — ✓ Upstream — ⋮ Review

Review

Review your selections before saving and deploying your API instance. You can also save the configuration and deploy it later.

[Edit](#)

Runtime

Runtime type	Mule Gateway	Edit
Proxy type	Basic Endpoint	

API

API name	SPS Customers API	Edit
API version	v1	
Asset version	1.0.0	
Conformance status	Not Validated ⓘ	

Downstream

Scheme	HTTP	Edit
--------	------	----------------------

Upstream

Upstream URL	https://spscustomersapi-m20wgk5sc6y6-1.usa-e2.cloudhub.io/api/v1/spc	Edit
--------------	--	----------------------

Imagen 43. Review de API nueva

Damos clic en **Save** y continuamos en **API Discovery** para conectar la API con nuestro proyecto del runtime manager con el API ID y Autodiscovery.

API Discovery

API Instances / SPS Customers API Actions ▾

❗ To complete the registration process, you need to connect this API to your Mule application using Autodiscovery. [Learn more](#)

Type RAML/OAS	Asset Version 1.0.0 (Latest)	Implementation URI ⓘ https://spscustomersapi-m20wgk5sc6y6-1.usa-e2.cloudhub.io/api/v1/spc	API Label ⓘ -	API Version v1	Edit
API Status Unregistered	Consumer Endpoint N/A	API Instance ID ⓘ 20898912	Instance Conformance ⓘ Not Validated		

Tags
[Add New Tag +](#)

Key Metrics

❗ If Key Metrics charts are not rendered correctly, make sure that Anypoint Monitoring is enabled. For more information, refer to this article: [Key Metrics charts in API Manager](#)

[View more metrics in Anypoint Monitoring dashboard](#) 🔄 📅 Last 24 hours ▾

Total Requests over Time ⓘ

Total Policy Violations over Time ⓘ

Imagen 44. API Summary de SPS Customers API ya creada

Para vincular nuestra aplicación desplegada con la entrada que acabamos de crear en el API Manager vamos a ir a **Anypoint Studio** y abrimos nuestros archivos de propiedades **local/dev.properties**. Una vez dentro, añadiremos una nueva propiedad llamada **api.id** en ambos archivos con el API ID que salga en donde está el recuadro de la imagen anterior.

```

spscustomersapi  global  local.properties x
1 http.listener.host=0.0.0.0
2 http.listener.port=8081
3 api.id=20898912
4
5 db.host=mudb.learn.mulesoft.com
6 db.port=3306
7 db.name=training

dev.properties x
1 http.listener.host=0.0.0.0
2 http.listener.port=8081
3 api.id=20898912
4
5 db.host=mudb.learn.mulesoft.com
6 db.port=3306
7 db.name=training
  
```

Imagen 45. API ID agregado a nuestros archivos de propiedades

Ahora abrimos nuestro archivo **global.xml** y vamos a la pestaña/vista de **Global Elements**, hacemos clic en **Create > API Autodiscovery**. Para vincular el API ID a nuestra propiedad nueva utilizamos la sintaxis para variables y seleccionamos el flujo **spscustomersapiflow**.

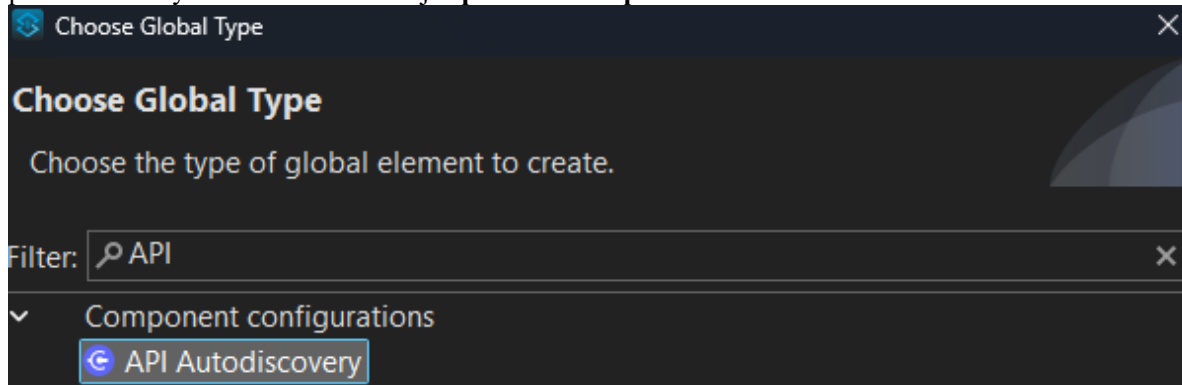


Imagen 46. Global Type en el menú

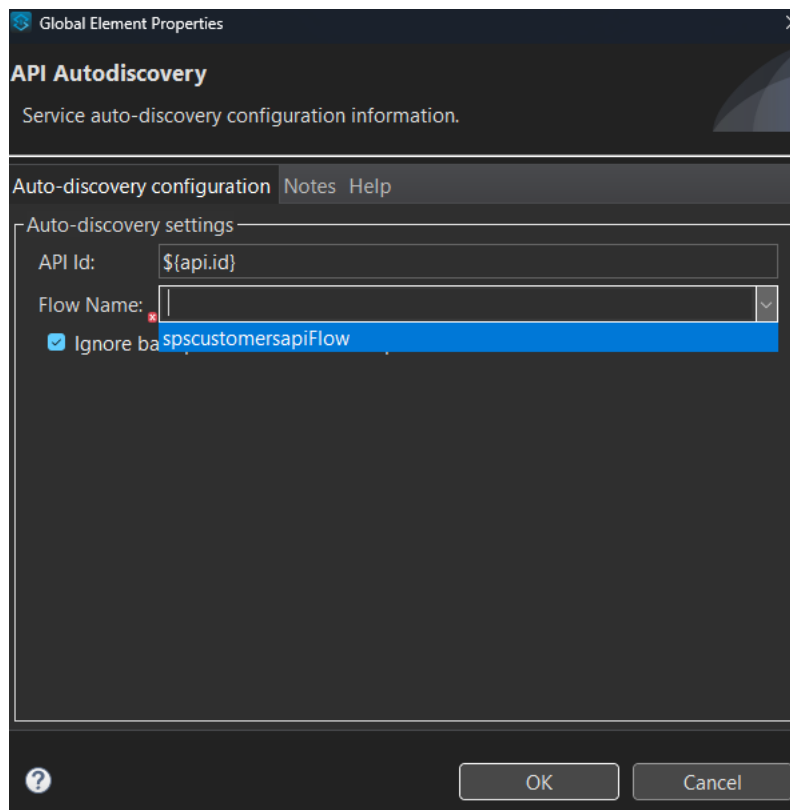


Imagen 47. Configuración del elemento global API Autodiscovery

Nos aseguramos de tener configurados tanto **Client ID** como **Client Secret** de acuerdo a como nos menciona la documentación de MuleSoft: [How to set up API Autodiscovery in Anypoint Studio | MuleSoft Developers](#) en el apartado “Test your application on your local computer”. Y probamos nuestra aplicación en local con **Run as > Run configurations...**

Finalmente cuando salga en consola **DEPLOYED** probamos con nuestro cliente rest llamando al endpoint de /customers.

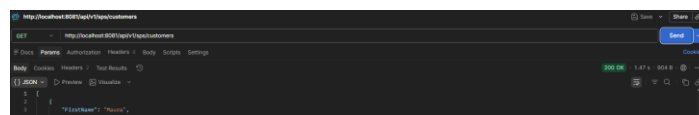


Imagen 48. Test con cliente REST de aplicación con Autodiscovery

Finalmente, nos queda desplegar a Cloudhub, para ello tenemos que abrir **mule-artifact.json** y añadir dos propiedades a la lista de **Secure Properties**.

```
"secureProperties": [
  "secure.key",
  "anypoint.platform.client_id",
  "anypoint.platform.client_secret"
],
{
  "minMuleVersion": "4.11.0",
  "secureProperties": [
    "secure.key",
    "anypoint.platform.client_id",
    "anypoint.platform.client_secret"
  ],
  "javaSpecificationVersions": [
    "17"
  ]
}
```

Imagen 49. Archivo mule-artifact.json

Repetimos el proceso de despliegue a Cloudhub, pero esta vez añadiendo dos propiedades.

1 **anypoint.platform.client_id**=<YOUR CLIENT ID>

2 **anypoint.platform.client_secret**=<YOUR CLIENT SECRET>

Deployment TargetIngressPropertiesMonitoringFirewall Rules

Table viewText view

Use properties to change the way your app behaves. [Learn more](#)

secure.key	MyMuleSoftKey	Protect	...
env	dev	Protect	...
anypoint.platform.client_id	protected value	🔒	...
anypoint.platform.client_secret	protected value	🔒	...
New Key	New Value		

Imagen 50. Propiedades añadidas antes de despliegue a Cloudhub con API Autodiscovery

Desplegamos la aplicación, una vez que haya terminado de desplegar en **Runtime Manager** deberíamos poder ver el estado de la API como **Active** desde **API Manager** y continuar con el **ClientID Police Enforcement** desde aquí sin necesidad de redesplegar.

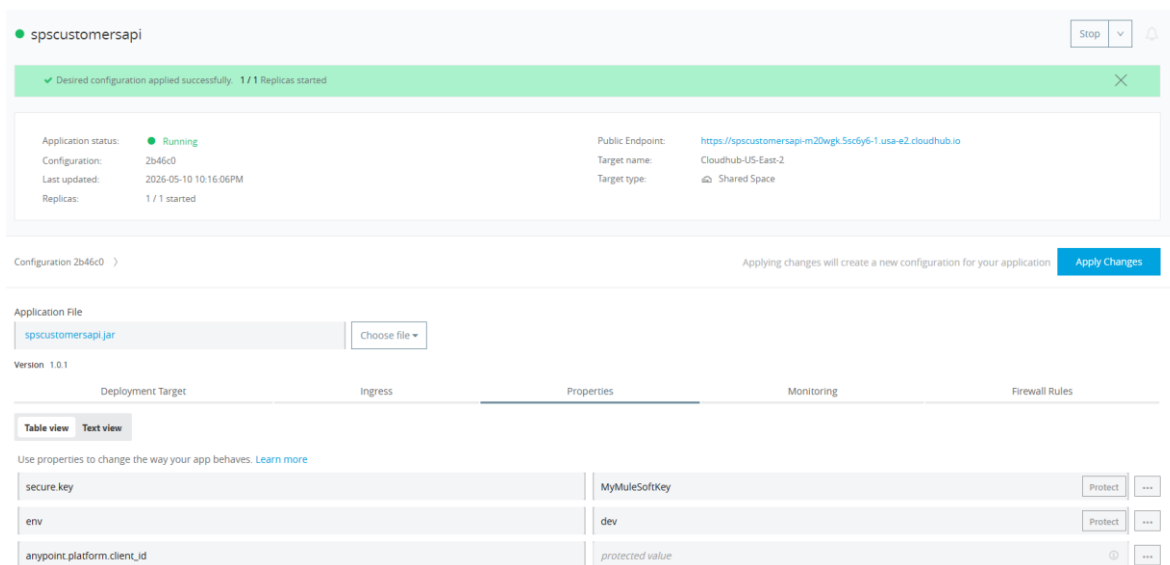


Imagen 51. SPS Customers API desplegada en Runtime Manager

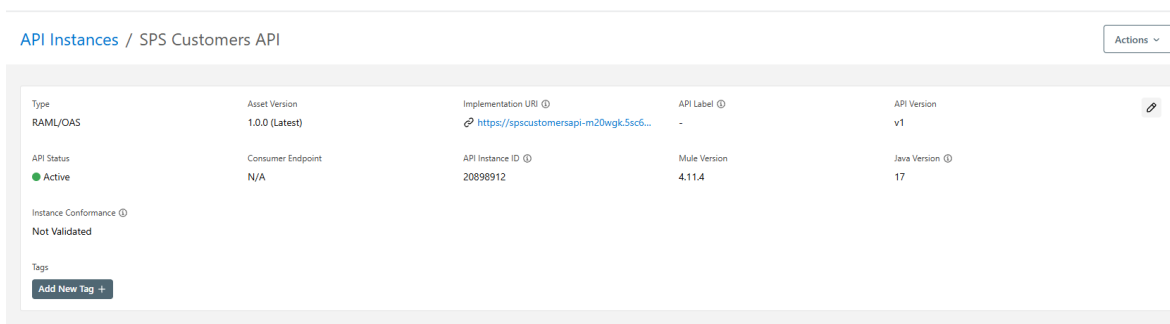


Imagen 52. SPS Customers API status desde API Manager al estar corriendo en Cloudhub

Probamos llamando a la API con el **Implementation URI** desde nuestro cliente REST.

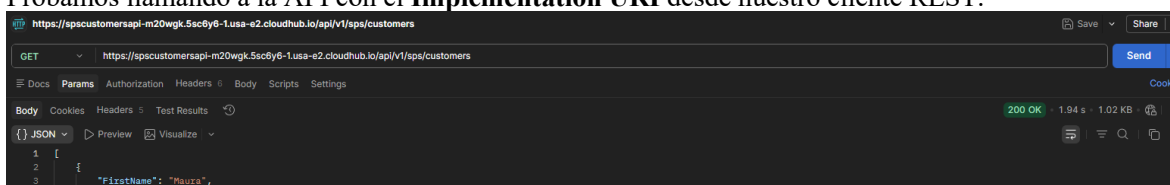


Imagen 53. Test de llamada a endpoint /customers después de despliegue en Cloudhub con API Autodiscovery

ClientID Police Enforcement

Finalmente, para añadir la política vamos a **API Manager** seleccionamos nuestra API **SPS Customers API** y hacemos clic en **Policies**.

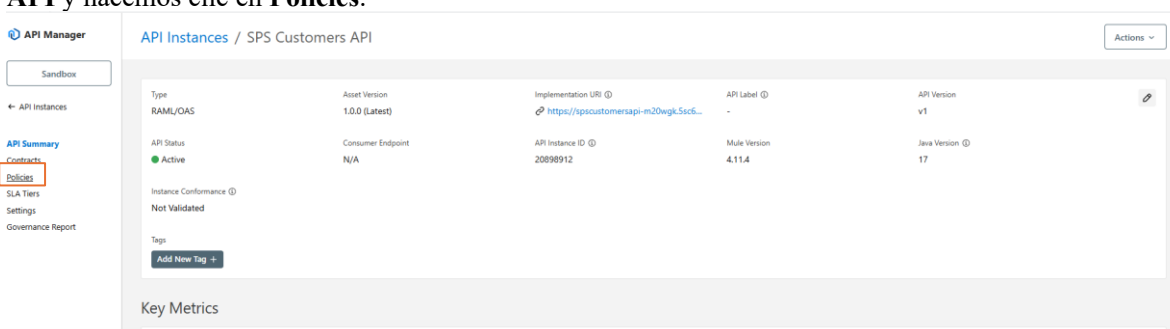


Imagen 54. Opción Policies dentro de la API en API Manager

Una vez en la página de **Policies** damos clic en **Add Policy** y buscamos **Client ID enforcement**, luego la seleccionamos y damos clic en **Next**.

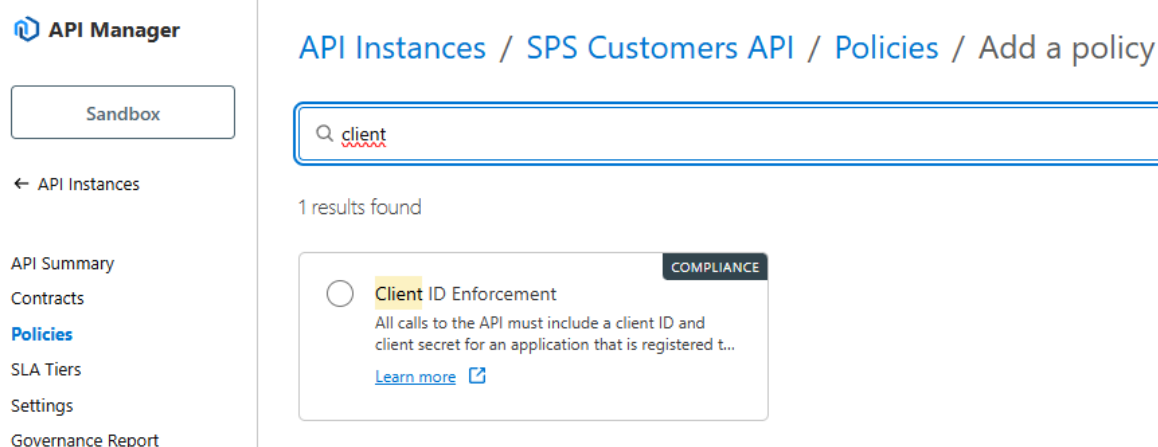


Imagen 55. Búsqueda de police Client ID Enforcement

Dentro de la configuración de la política seleccionaremos **HTTP Basic Authentication Header** y aplicamos configuraciones a todos los métodos y recursos de la API seleccionando **all API methods & resources**. Luego damos clic en **Apply**.

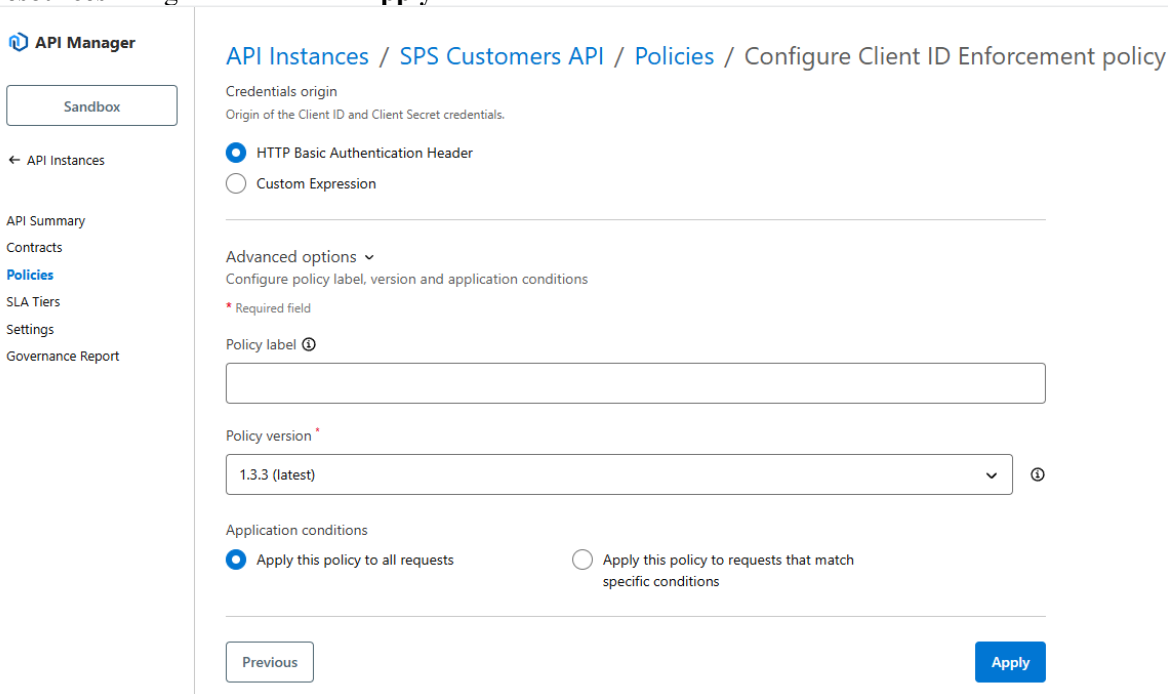


Imagen 56. Aplicación de política de Client ID Enforcement

Ahora probamos en nuestro cliente REST que se haya aplicado correctamente, reutilizando la llamada al endpoint de **/customers**. Deberíamos recibir un código **401 Unauthorized**.

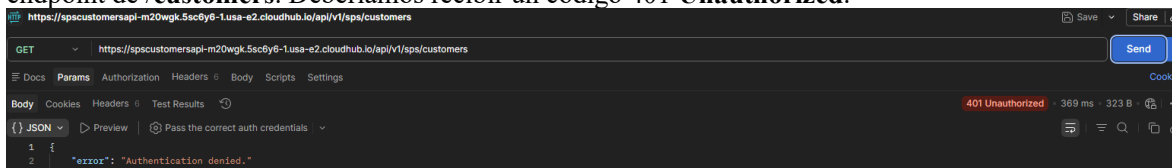


Imagen 57. Test de falla de llamada a endpoint /customers